

```
#include "SparkFun_SHTC3.h" // Click here to get the library:  
http://librarymanager/All#SparkFun\_SHTC3
```

```
SHTC3 mySHTC3;           // Declare an instance of the SHTC3 class
```

```
void setup() {
```

```
    Serial.begin(9600);           // Begin Serial  
    while(Serial == false){};    // Wait for  
the serial connection to start up  
    Serial.println("SHTC3 Example 1 - Basic Readings"); // Title  
    Wire.begin();  
    Serial.print("Beginning sensor. Result = ");        // Most SHTC3  
functions return a variable of the type "SHTC3_Status_TypeDef" to indicate  
the status of their execution  
    errorDecoder(mySHTC3.begin()); // To start  
the sensor you must call "begin()", the default settings use Wire (default  
Arduino I2C port)  
    Serial.println();  
    Serial.println("\n\n");  
    Serial.println("Waiting for 5 seconds so you can read this info ^^^");  
  
    delay(5000); // Give time  
to read the welcome message and device ID.  
}
```

```
void loop() {  
    SHTC3_Status_TypeDef result = mySHTC3.update(); // Call  
"update()" to command a measurement, wait for measurement to complete, and  
update the RH and T members of the object  
    printInfo(); // This  
function is used to print a nice little line of info to the serial port  
    delay(190); // Delay for  
the data rate you want - note that measurements take ~10 ms so the fastest  
data rate is 100 Hz (when no delay is used)  
}
```

```
////////////////////////////////////  
// Utility Functions //  
////////////////////////////////////
```

```
void printInfo()  
{  
    if(mySHTC3.lastStatus == SHTC3_Status_Nominal) // You can  
also assess the status of the last command by checking the ".lastStatus"  
member of the object  
    {  
        Serial.print("RH = ");
```

```

        Serial.print(mySHTC3.toPercent()); //
        "toPercent" returns the percent humidity as a floating point number
        Serial.print("%, T = ");
        Serial.print(mySHTC3.toDegF()); // "toDegF"
and "toDegC" return the temperature as a floating point number in deg F
and deg C respectively
        Serial.println(" deg F");
    }
    else
    {
        Serial.print("Update failed, error: ");
        errorDecoder(mySHTC3.lastStatus);
        Serial.println();
    }
}

void errorDecoder(SHTC3_Status_TypeDef message)
// The errorDecoder function prints "SHTC3_Status_TypeDef" results in a
human-friendly way
{
    switch(message)
    {
        case SHTC3_Status_Nominal : Serial.print("Nominal"); break;
        case SHTC3_Status_Error : Serial.print("Error"); break;
        case SHTC3_Status_CRC_Fail : Serial.print("CRC Fail"); break;
        default : Serial.print("Unknown return code"); break;
    }
}

```